BÀI 5: PHÂN TÍCH VÀ KIỂM THỬ BẢO MẬT

Bài thực hành này trang bị cho sinh viên nền tảng vững chắc về hai kỹ thuật trọng yếu: Static Analysis Security Testing (SAST) và Dynamic Analysis Security Testing (DAST). Với SAST, sinh viên học cách phân tích mã nguồn qua cây cú pháp trừu tượng, phát hiện lỗ hổng tiềm ẩn trước khi phần mềm chạy. DAST giúp sinh viên quan sát hành vi ứng dụng khi thực thi, từ giám sát bộ nhớ đến phân tích mạng. Qua các bài thực hành, sinh viên được trải nghiệm toàn diện quy trình kiểm thử bảo mật hiện đại, phát triển tư duy phòng thủ chủ động và nâng cao năng lực phát hiện – khắc phục lỗi bảo mật trong phát triển phần mềm.

5.1 STATIC ANALYSIS SECURITY TESTING

5.1.1 Cây cú pháp trừu tượng

AST (Abstract Syntax Tree – Cây cú pháp trừu tượng) là biểu diễn dạng cây của cấu trúc cú pháp mã nguồn, trong đó mỗi nút đại diện cho một cấu trúc ngôn ngữ như biểu thức, lệnh gán, hàm, hoặc điều kiện. AST bỏ qua các chi tiết không cần thiết như dấu ngoặc hoặc thụt đầu dòng, chỉ giữ lại cấu trúc logic của chương trình. Trong kiểm thử bảo mật tĩnh (SAST), AST cho phép phân tích cấu trúc mã mà không cần chạy chương trình. Việc duyệt AST giúp xác định:

- Các mẫu mã nguy hiểm (gọi eval, thao tác trực tiếp với input người dùng).
- Các cấu trúc thiếu an toàn (ví dụ: thiếu kiểm tra xác thực, vòng lặp vô hạn).
- Những hành vi tiềm ẩn rủi ro khi mã được triển khai thực tế.

Chúng ta có thể sử dụng thư viện `ast` trong Python để:

- Phân tích cây cú pháp của một đoạn mã.
- Xây dựng trình kiểm tra lỗ hổng theo rule.

- Phát hiện hành vi nguy hiểm hoặc chưa tuân thủ chuẩn bảo mật.

Ví dụ:

```
import ast

code = "eval(input('Nhập lệnh: '))"

tree = ast.parse(code)

for node in ast.walk(tree):

    if isinstance(node, ast.Call) and getattr(node.func, 'id', '') ==
'eval':

        print("Cảnh báo: Sử dụng eval có thể gây nguy hiểm.")
```

5.1.2 Kỹ thuật Pattern Matching

Pattern Matching là kỹ thuật tìm kiếm các cấu trúc mã cụ thể (patterns) thường liên quan đến lỗ hổng bảo mật. Thay vì chỉ tìm từ khóa, pattern matching phân tích ngữ cảnh và cấu trúc để xác định hành vi đáng ngờ như SQL Injection, XSS, hoặc Insecure Deserialization. Việc phát hiện sớm các mẫu mã nguy hiểm giúp:

- Ngăn chặn lỗi bảo mật trước khi mã được triển khai.
- Xây dựng các rule kiểm tra bảo mật có thể tái sử dụng.
- Hiểu được "chân dung" điển hình của các lỗ hổng phổ biến.

Phân loại pattern:

- Literal pattern: Tìm trực tiếp chuỗi cụ thể (ví dụ: 'os.system', 'eval').
- Structural pattern: Dựa vào cấu trúc mã (dạng biểu thức, cây AST).
- Semantic pattern: Dựa vào hành vi tiềm ẩn (ví dụ: nhập người dùng → xử lý → thực thi).

Chúng ta có thể viết các pattern rule kiểm tra trong công cụ tự xây dựng bằng cách:

- Duyệt qua AST để tìm những call function có rủi ro.
- Kiểm tra mối quan hệ giữa các node (input → xử lý → output).
- Gắn thẻ các đoạn mã nghi vấn để báo cáo.

Ví dụ:

```
import ast
class VulnPatternMatcher(ast.NodeVisitor):
    def visit_Call(self, node):
        if isinstance(node.func, ast.Name) and node.func.id == "eval":
            print(f"Cảnh báo: phát hiện 'eval' tại dòng {node.lineno}")
        self.generic_visit(node)
tree = ast.parse("user_input = input(); eval(user_input)")
VulnPatternMatcher().visit(tree)
```

5.1.3 Đánh giá chất lượng mã nguồn

Code Quality Metrics là tập hợp các chỉ số dùng để đo lường mức độ "sạch", "ổn định" và "an toàn" của mã nguồn. Trong ngữ cảnh bảo mật, các chỉ số này giúp xác định liệu mã có dễ bị lỗi, khó bảo trì, hoặc tiềm ẩn lỗ hổng hay không. Một số chỉ số phổ biến:

- **Cyclomatic Complexity** (Độ phức tạp vòng lặp): Số lượng nhánh điều kiện trong một hàm. Độ phức tạp cao → khó kiểm thử → dễ lỗi.
- **Function Length** (Độ dài hàm): Hàm quá dài thường chứa logic chồng chéo, dễ bỏ sót lỗ hổng.
- **Nested Depth** (Độ sâu lồng nhau): Cấu trúc if/else hoặc loop lồng nhiều lớp → làm mờ luồng điều khiển → tăng rủi ro logic sai lệch.
- **Dependency Count** (Số lượng phụ thuộc): Sử dụng nhiều thư viện bên ngoài → tăng bề mặt tấn công.
- **Security Smell Density**: Tần suất xuất hiện của các đoạn mã có "mùi nguy hiểm" như eval(), hardcoded credentials, hoặc os.system.

Ý nghĩa với bảo mật:

- Cảnh báo sớm các vị trí dễ phát sinh lỗ hổng.
- Ưu tiên kiểm thử và audit những phần mã rủi ro cao.
- Cải thiện khả năng bảo trì và audit mã trong dài hạn.

Ví dụ:

```
def cyclomatic_complexity(func_node):
    complexity = 1
```

```
    for node in ast.walk(func_node):
        if isinstance(node, (ast.If, ast.For, ast.While, ast.Try)):
            complexity += 1
    return complexity
```

5.2 THỰC HÀNH: SAST-TOOL

5.2.1 Yêu cầu

Xây dựng công cụ phân tích bảo mật tĩnh (SAST) cho mã nguồn Python, bao gồm các thành phần sau:

- *ASTAnalyzer*: Phân tích cây cú pháp trừu tượng của mã Python để phát hiện các vấn đề bảo mật.
- *VulnPatternMatcher*: Ánh xạ và nhận diện lỗ hổng dựa trên mẫu tùy chỉnh.
- *SecurityRuleEngine*: Hệ thống luật bảo mật có thể mở rộng và cài cắm linh hoạt.
- *CodeQualityChecker*: Đánh giá chất lượng mã với trọng tâm là các chỉ số liên quan đến bảo mật.
- *SarifReporter*: Xuất báo cáo kết quả phân tích theo định dạng chuẩn SARIF (Static Analysis Results Interchange Format).

5.2.2 Hướng dẫn thực hành

- Tạo folder "`sast-tool`". Trong folder này tạo các folder "`examples`", "`rules`", "`sast_tool`", "`tests`" và file "`requirements.txt`".
- File "`requirements.txt`":



- Cài các gói cần thiết bằng lệnh pip:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\sast-tool> pip install -r .\requirements.txt
Requirement already satisfied: flake8 in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (from -r .\requirem
Requirement already satisfied: sarif-om in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (from -r .\requir
Requirement already satisfied: jsonpickle in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (from -r .\requ
Requirement already satisfied: mccabe<0.8.0,>=0.7.0 in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (from
Requirement already satisfied: pycodestyle<2.15.0,>=2.14.0 in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-package
Requirement already satisfied: pyflakes<3.5.0,>=3.4.0 in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (fr
Requirement already satisfied: attrs in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (from sarif-om->-r .
Requirement already satisfied: pbr in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (from sarif-om->-r .\r
Requirement already satisfied: setuptools in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (from pbr->sari
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\sast-tool>
```

- Code cho file "`examples/vulnerable_01.py`":

sast-tool > examples > vulnerable_01.py > ...

```python
password = "123456"  # Hardcoded password

import pickle  # Có thể gây nguy hiểm

def unsafe_func(data):
    return eval(data)  # Rất nguy hiểm, eval dùng tùy ý

def safe_func():
    print("Hello world!")

if __name__ == "__main__":
    unsafe_func("2 + 2")
    safe_func()

```

- Code cho file "`examples/vulnerable_02.py`":

sast-tool > examples > vulnerable_02.py > dangerous_eval

```python
def dangerous_eval(user_input):
    # Nguy hiểm vì eval không kiểm soát đầu vào
    return eval(user_input)

def read_file(filename):
    with open(filename, 'r') as f:
        data = f.read()
    return data

def execute_code(code):
    exec(code)

if __name__ == "__main__":
    user_code = "print('Hello from eval')"
    dangerous_eval(user_code)

```

- Code cho file "`rules/default_rules.json`":

sast-tool > rules > {} default_rules.json > ...

```json
[
  {"type": "Use of eval", "severity": "high"},
  {"type": "Pattern Match", "severity": "medium"},
  {"type": "Lint Issue", "severity": "low"}
]

```

- Tạo file "`sast_tool/__init__.py`".

- Code cho file "`sast_tool/ast_analyzer.py`":

sast-tool > sast_tool > ast_analyzer.py > ...

```python
import ast

class ASTAnalyzer:
    def analyze(self, code: str):
        tree = ast.parse(code)
        issues = []
        for node in ast.walk(tree):
            if isinstance(node, ast.Call) and getattr(node.func, 'id', '') == 'eval':
                issues.append({
                    'type': 'Use of eval',
                    'message': 'Use of eval() is dangerous and should be avoided.',
                    'lineno': node.lineno,
                    'col_offset': node.col_offset
                })
        return issues

```

- Code cho file "`sast_tool/code_quality_checker.py`":

sast-tool > sast_tool > code_quality_checker.py > CodeQualityChecker > check

```python
import subprocess

class CodeQualityChecker:
    def check(self, filepath):
        issues = []
        result = subprocess.run(["flake8", filepath],
                                capture_output=True, text=True)
        for line in result.stdout.splitlines():
            if line:
                parts = line.split(":")
                if len(parts) >= 4:
                    # kiểm tra chắc chắn phần line và col là số
                    try:
                        lineno = int(parts[1])
                        col_offset = int(parts[2])
                    except ValueError:
                        continue
                    issues.append({
                        'type': 'Lint Issue',
                        'message': line,
                        'lineno': lineno,
                        'col_offset': col_offset
                    })
        return issues

```

- Code cho file "`sast_tool/sarif_reporter.py`":

sast-tool > sast_tool > sarif_reporter.py > ...

```python
from sarif_om import SarifLog, Run, Tool, ToolComponent, Result, Message
from sarif_om import ArtifactLocation, Location, PhysicalLocation, Region
import jsonpickle

class SarifReporter:
    def generate_report(self, issues, output_file='report.sarif',
                        analyzed_file="input.py"):
        results = []
        for i in issues:
            results.append(Result(
                message=Message(text=i['message']),
                locations=[Location(
                    physical_location=PhysicalLocation(
                        artifact_location=ArtifactLocation(uri=analyzed_file),
                        region=Region(start_line=i['lineno'],
                                      start_column=i['col_offset'] + 1)
                    )
                )]
            ))
```

```
20
21          run = Run(
22              tool=Tool(driver=ToolComponent(name="PySAST", version="1.0.0")),
23              results=results
24          )
25          log = SarifLog(schema_uri="https://json.schemastore.org/sarif-2.1.0.json",
26                         version="2.1.0", runs=[run])
27
28          with open(output_file, 'w') as f:
29              f.write(jsonpickle.encode(log, unpicklable=False, indent=2))
30
```

- Code cho file "`sast_tool/security_rule_engine.py`":

sast-tool > sast_tool > security_rule_engine.py > ...

```
1  import json
2
3  class SecurityRuleEngine:
4      def __init__(self, rules_file='rules/default_rules.json'):
5          with open(rules_file) as f:
6              self.rules = json.load(f)
7
8      def evaluate(self, ast_issues, pattern_issues):
9          issues = []
10         for issue in ast_issues + pattern_issues:
11             for rule in self.rules:
12                 if rule['type'] in issue['type']:
13                     issue['severity'] = rule.get('severity', 'medium')
14                     issues.append(issue)
15         return issues
16
```

- Code cho file "`sast_tool/vuln_pattern_matcher.py`":

sast-tool > sast_tool > vuln_pattern_matcher.py > ...

```
1  import re
2
3  class VulnPatternMatcher:
4      def match(self, code: str):
5          patterns = [
6              {'pattern': r'password\s*=\s*".*"',
7               'message': 'Hardcoded password detected.'},
8              {'pattern': r'import\s+pickle',
9               'message': 'Usage of pickle module can be unsafe.'}
10         ]
```

```
        issues = []
        for p in patterns:
            for match in re.finditer(p['pattern'], code):
                issues.append({
                    'type': 'Pattern Match',
                    'message': p['message'],
                    'lineno': code[:match.start()].count('\n') + 1,
                    'col_offset': match.start() - code.rfind('\n', 0, match.start()) - 1
                })
        return issues

```

- Code cho file "`sast_tool/main.py`":

sast-tool > sast_tool > main.py > ...

```python
import sys
from sast_tool.ast_analyzer import ASTAnalyzer
from sast_tool.vuln_pattern_matcher import VulnPatternMatcher
from sast_tool.security_rule_engine import SecurityRuleEngine
from sast_tool.code_quality_checker import CodeQualityChecker
from sast_tool.sarif_reporter import SarifReporter

def main(file_path):
    with open(file_path) as f:
        code = f.read()

    ast_issues = ASTAnalyzer().analyze(code)
    pattern_issues = VulnPatternMatcher().match(code)
    quality_issues = CodeQualityChecker().check(file_path)

    rule_engine = SecurityRuleEngine()
    final_issues = rule_engine.evaluate(ast_issues, pattern_issues) + quality_issues

    SarifReporter().generate_report(final_issues, analyzed_file=file_path)
    print(f'''Analysis complete.
          Report saved to report.sarif with {len(final_issues)} issues.''')

if __name__ == '__main__':
    if len(sys.argv) != 2:
        print("Usage: python -m sast_tool.main <file_to_analyze.py>")
        sys.exit(1)
    main(sys.argv[1])

```

- Tạo file "`tests/__init__.py`".

- Code cho file "`tests/test_ast_analyzer.py`":

sast-tool > tests > test_ast_analyzer.py > ...

```python
import unittest
from sast_tool.ast_analyzer import ASTAnalyzer

class TestASTAnalyzer(unittest.TestCase):
    def test_eval_detection(self):
        code = "eval('2+2')"
        analyzer = ASTAnalyzer()
        issues = analyzer.analyze(code)
        self.assertTrue(any(i['type'] == 'Use of eval' for i in issues))

```

- Code cho file "`tests/test_code_quality_checker.py`":

sast-tool > tests > test_code_quality_checker.py > ...

```python
import unittest
import tempfile
from sast_tool.code_quality_checker import CodeQualityChecker

class TestCodeQualityChecker(unittest.TestCase):
    def test_flake8_detection(self):
        with tempfile.NamedTemporaryFile(suffix=".py",
                                         mode="w", delete=True) as tmp:
            tmp.write("import os\n\n\n")
            tmp.flush()
            checker = CodeQualityChecker()
            issues = checker.check(tmp.name)
            self.assertTrue(all(i['type'] == 'Lint Issue' for i in issues)
                            or len(issues) == 0)

```

- Code cho file "`tests/test_sarif_reporter.py`":

sast-tool > tests > test_sarif_reporter.py > ...

```python
import unittest
import os
from sast_tool.sarif_reporter import SarifReporter

class TestSarifReporter(unittest.TestCase):
    def test_generate_report(self):
        issues = [{
            'type': 'Test',
            'message': 'Test message',
            'lineno': 1,
            'col_offset': 0
        }]
```

```
        reporter = SarifReporter()
        output_file = "test_report.sarif"
        reporter.generate_report(issues, output_file)
        self.assertTrue(os.path.exists(output_file))
        os.remove(output_file)

```

- Code cho file "`tests/test_security_rule_engine.py`":

sast-tool > tests > test_security_rule_engine.py > ...

```python
import unittest
import json
from sast_tool.security_rule_engine import SecurityRuleEngine

class TestSecurityRuleEngine(unittest.TestCase):
    def setUp(self):
        # Create temporary rule file
        self.rules = [
            {"type": "Use of eval", "severity": "high"},
            {"type": "Pattern Match", "severity": "medium"}
        ]
        with open("rules/temp_rules.json", "w") as f:
            json.dump(self.rules, f)
        self.engine = SecurityRuleEngine("rules/temp_rules.json")

    def tearDown(self):
        import os
        os.remove("rules/temp_rules.json")

    def test_evaluate(self):
        ast_issues = [{"type": "Use of eval", "message": "msg",
                       "lineno": 1, "col_offset": 0}]
        pattern_issues = [{"type": "Pattern Match", "message":
            "msg", "lineno": 2, "col_offset": 1}]
        results = self.engine.evaluate(ast_issues, pattern_issues)
        self.assertEqual(len(results), 2)
        self.assertEqual(results[0]['severity'], "high")
        self.assertEqual(results[1]['severity'], "medium")

```

- Code cho file "`tests/test_vuln_pattern_matcher.py`":

```
sast-tool > tests > test_vuln_pattern_matcher.py > ...
1  import unittest
2  from sast_tool.vuln_pattern_matcher import VulnPatternMatcher
3
4  class TestVulnPatternMatcher(unittest.TestCase):
5      def test_hardcoded_password(self):
6          code = 'password = "secret"'
7          matcher = VulnPatternMatcher()
8          issues = matcher.match(code)
9          self.assertTrue(any(i['type'] == 'Pattern Match' for i in issues))
10
11     def test_pickle_import(self):
12         code = 'import pickle'
13         matcher = VulnPatternMatcher()
14         issues = matcher.match(code)
15         self.assertTrue(any(i['type'] == 'Pattern Match' for i in issues))
16
```

- Chạy Unit Test:

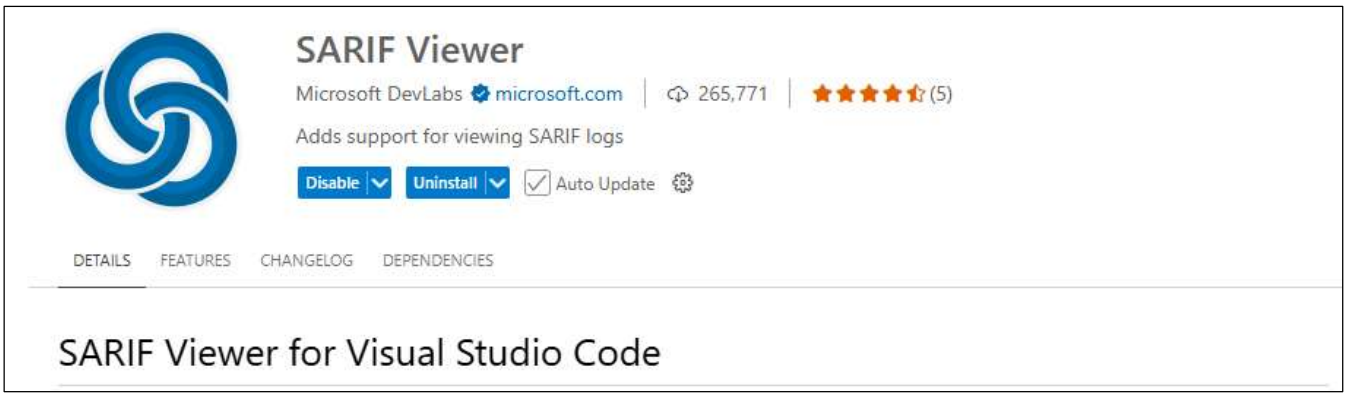
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\sast-tool> python -m unittest discover tests
......
----------------------------------------------------------------------
Ran 6 tests in 0.373s

OK
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\sast-tool>
```

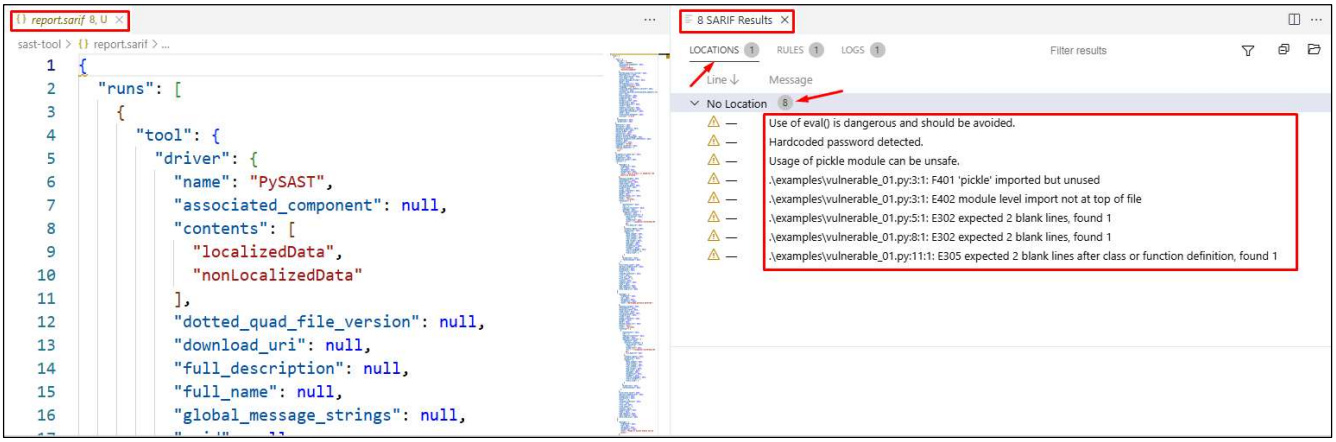
- Kiểm tra thử 1 file code không an toàn:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\sast-tool> python -m sast_tool.main .\examples\vulnerable_01.py
Analysis complete.
        Report saved to report.sarif with 8 issues
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\sast-tool>
```

- Vào mục Extensions của Visual Studio Code và cài "SARIF Viewer".



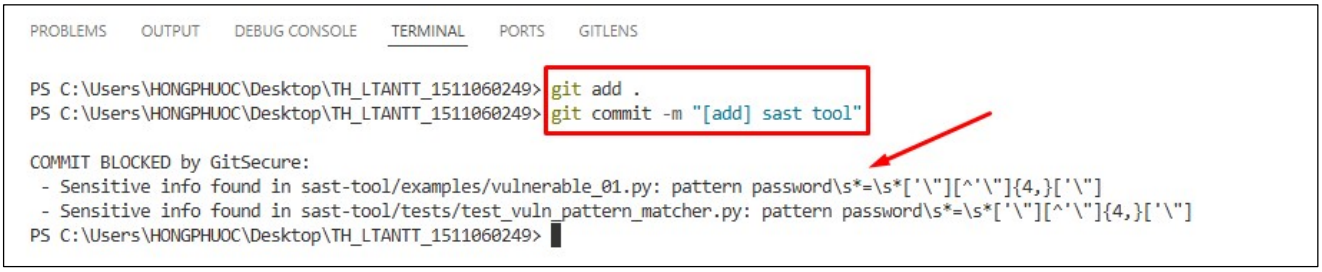
- Mở file "`report.sarif`" để kiểm tra kết quả:



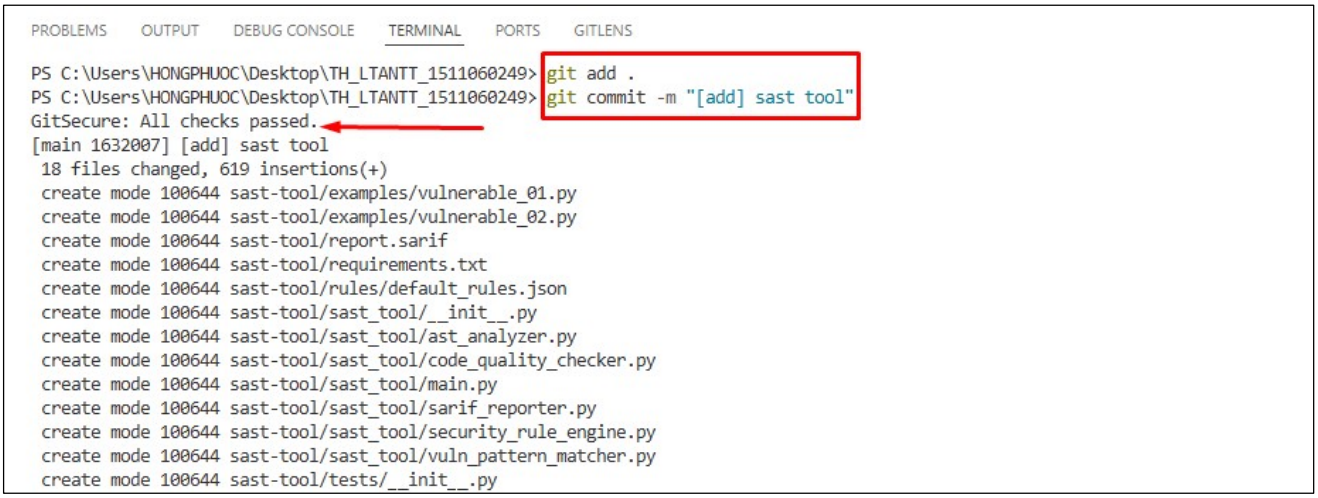
- Kiểm tra lại với một file code không an toàn khác:



- Commit code, push code lên remote repo:



- Xoá các thông tin nhạy cảm, sau đó commit lại và push code.





5.3 DYNAMIC ANALYSIS SECURITY TESTING

5.3.1 Runtime Security Testing

Runtime Security Testing (RST) là quá trình theo dõi và phân tích hành vi của phần mềm trong khi nó đang chạy để phát hiện các hoạt động nguy hiểm, lỗi bảo mật tiềm ẩn hoặc hành vi bất thường. Không giống kiểm thử tĩnh (Static Analysis) vốn làm việc trên mã nguồn, RST cho phép phát hiện các lỗ hổng chỉ xuất hiện trong môi trường thực thi. Mục tiêu của RST:

- Giám sát tương tác thời gian thực giữa ứng dụng và hệ điều hành, mạng, bộ nhớ, và tài nguyên hệ thống.
- Phát hiện hành vi nguy hiểm như:
  - Gọi hàm hệ thống nguy hiểm (vd: os.system, eval).
  - Tấn công injection xảy ra trong runtime (SQLi, RCE, Command Injection).
  - Lỗi logic chỉ xảy ra khi chạy (race condition, insecure state).
- Đánh giá mức độ tuân thủ các chính sách bảo mật đã định nghĩa.

Các thành phần chính trong RST Platform:

Thành phần	Mô tả chức năng
RuntimeMonitor	Ghi nhận và ghi log các hành vi quan trọng trong suốt quá trình thực thi của ứng dụng như I/O, gọi API, tiến trình con...
MemoryAnalyzer	Kiểm tra các vùng bộ nhớ cấp phát động nhằm phát hiện rò rỉ bộ nhớ, truy cập sai vùng nhớ hoặc ghi tràn buffer.
NetworkTrafficAnalyzer	Phân tích lưu lượng mạng của ứng dụng: kết nối mở, dữ liệu gửi/nhận, tên miền truy cập, phát hiện C2 hoặc exfiltration.

Thành phần	Mô tả chức năng
SecurityBehaviorChecker	So khớp hành vi runtime với các mẫu (rule/heuristic) đã học để phát hiện hành vi nghi ngờ như keylogging, tải shellcode, tự nhân bản.
VirtualSandbox	Cung cấp môi trường cách ly giúp chạy thử phần mềm độc hại mà không làm ảnh hưởng tới hệ điều hành thật.

5.3.2 Phân tích hành vi

Behavioral Analysis (Phân tích hành vi) là kỹ thuật quan sát và đánh giá các hành vi mà một chương trình hoặc tiến trình thể hiện trong suốt quá trình thực thi. Khác với phân tích chữ ký (signature-based), phân tích hành vi không cần biết trước mẫu mã độc mà thay vào đó phát hiện các hành vi bất thường hoặc nguy hiểm thông qua giám sát thời gian thực và đối chiếu với mô hình hành vi hợp lệ.

Mục tiêu của Behavioral Analysis:

- Phát hiện phần mềm độc hại chưa từng biết (unknown malware).
- Nhận diện hành vi gian lận, vượt quyền hoặc tấn công đang diễn ra.
- Theo dõi dòng chảy hành vi của ứng dụng để phát hiện sự lệch chuẩn.
- Tăng khả năng phát hiện các kỹ thuật né tránh sandbox.

Các kỹ thuật phân tích hành vi phổ biến:

Kỹ thuật	Mô tả
Rule-based detection	So khớp hành vi runtime với các luật định nghĩa sẵn. Ví dụ: "nếu tiến trình đọc file /etc/passwd và mở socket ra ngoài → cảnh báo."
Heuristic Analysis	Áp dụng một tập các tiêu chí (heuristics) để đánh giá tính nguy hiểm. Không cần chính xác tuyệt đối nhưng phản ứng nhanh.
Anomaly Detection	Sử dụng thống kê hoặc AI để xác định các hành vi **khác thường** so với baseline của hệ thống hoặc người dùng.
Machine Learning-based detection	Trích xuất đặc trưng hành vi (feature extraction) và huấn luyện mô hình học máy để phân biệt "bình thường" và "nghi ngờ."

5.3.3 Phân tích trong môi trường hộp cát

Sandbox Analysis (Phân tích trong môi trường hộp cát) là kỹ thuật thực thi phần mềm trong một môi trường ảo hoá hoặc cô lập hoàn toàn nhằm quan sát hành vi mà không gây ảnh hưởng đến hệ thống thật. Sandbox cho phép các công cụ bảo mật phân tích phần mềm (đặc biệt là phần mềm chưa xác định rõ nguồn gốc) một cách an toàn và có kiểm soát.

Các loại Sandbox:

Loại	Mô tả	Ví dụ
VM-based	Dùng máy ảo (VirtualBox, QEMU, VMware) để mô phỏng OS thật	Cuckoo Sandbox
Container-based	Dùng Docker/LXC – nhanh hơn, nhẹ hơn nhưng ít cô lập hơn	Firejail, Sysdig Falco
Bare-metal	Chạy trên máy thật, giám sát bằng phần cứng hoặc hypervisor	Joe Sandbox Bare Metal
In-memory/emulator	Dùng trình mô phỏng CPU/memory (QEMU, Unicorn) để quan sát chi tiết cấp thấp	PANDA, Intel PIN

5.4 THỰC HÀNH: DAST-TOOL

5.4.1 Yêu cầu

Xây dựng nền tảng phân tích động (Dynamic Analysis Platform). Hệ thống gồm các thành phần sau:

- *RuntimeMonitor*: Giám sát hành vi của ứng dụng trong quá trình thực thi.
- *MemoryAnalyzer*: Phát hiện các vấn đề rò rỉ bộ nhớ (memory leak) và tràn bộ nhớ (overflow) cơ bản.
- *NetworkTrafficAnalyzer*: Phân tích các luồng giao tiếp mạng trong thời gian thực.
- *SecurityBehaviorChecker*: Phát hiện các hành vi bất thường hoặc nghi ngờ có tính độc hại trong quá trình chạy ứng dụng.
- *VirtualSandbox*: Môi trường thực thi cách ly (sandbox) nhằm đảm bảo ứng dụng được kiểm tra không ảnh hưởng đến hệ thống thật.

5.4.2 Hướng dẫn thực hành

- Tạo folder "`dast-tool`". Trong folder này tạo folder "`tests`" và file "`__init__.py`". File "`requirements.txt`":

dast-tool > requirements.txt

```
1  psutil
2  Flask
```

- Cài các gói cần thiết qua lệnh pip:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS

PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\dast-tool> pip install -r .\requirements.txt
Requirement already satisfied: psutil in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (from -r .\require
Requirement already satisfied: Flask in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (from -r .\requirem
Requirement already satisfied: Werkzeug>=2.3.7 in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (from Fla
Requirement already satisfied: Jinja2>=3.1.2 in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (from Flask
Requirement already satisfied: itsdangerous>=2.1.2 in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (from
Requirement already satisfied: click>=8.1.3 in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (from Flask-
Requirement already satisfied: blinker>=1.6.2 in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (from Flas
Requirement already satisfied: colorama in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (from click>=8.1
```

- Code cho file "`memory_analyzer.py`":

dast-tool > memory_analyzer.py > ...

```python
import gc

class MemoryAnalyzer:
    def check_memory_leaks(self):
        gc.collect()
        leaks = [obj for obj in gc.garbage if hasattr(obj, '__del__')]
        return leaks

    def check_overflow(self, value):
        try:
            if isinstance(value, int) and abs(value) > 2**63:
                return True
        except:
            return False
        return False

```

- Code cho file "network_traffic_analyzer.py":

dast-tool > network_traffic_analyzer.py > ...

```python
import psutil

class NetworkTrafficAnalyzer:
    def get_network_stats(self):
        return psutil.net_io_counters(pernic=True)

```

- Code cho file "`runtime_monitor.py`":

dast-tool > runtime_monitor.py > ...

```python
import time
import psutil

class RuntimeMonitor:
    def __init__(self, pid):
        self.pid = pid
        self.process = psutil.Process(pid)

    def get_cpu_usage(self):
        return self.process.cpu_percent(interval=1.0)

    def get_memory_usage(self):
        return self.process.memory_info().rss

    def get_open_files(self):
        return self.process.open_files()

```

- Code cho file "security_behavior_checker.py":

dast-tool > security_behavior_checker.py > ...

```python
import os

class SecurityBehaviorChecker:
    def suspicious_calls(self):
        suspicious = []
        syscalls = ["os.system", "eval", "exec"]
        try:
            with open(__file__, 'r') as f:
                content = f.read()
                for call in syscalls:
                    if call in content:
                        suspicious.append(call)
        except:
            pass
        return suspicious

```

- Code cho file "`virtual_sandbox.py`":

dast-tool > virtual_sandbox.py > ...

```python
import subprocess
import tempfile
import os

class VirtualSandbox:
    def __init__(self):
        self.temp_dir = tempfile.mkdtemp()

    def run_isolated(self, command):
        result = subprocess.run(command, cwd=self.temp_dir,
                                capture_output=True, shell=True)
        return result.stdout.decode(), result.stderr.decode()

```

- Code cho file "`main.py`":

dast-tool > main.py > ...

```python
import os
from runtime_monitor import RuntimeMonitor
from memory_analyzer import MemoryAnalyzer
from network_traffic_analyzer import NetworkTrafficAnalyzer
from security_behavior_checker import SecurityBehaviorChecker
from virtual_sandbox import VirtualSandbox

if __name__ == "__main__":
    pid = os.getpid()
    print("[+] Monitoring current process:", pid)

    rm = RuntimeMonitor(pid)
    print("CPU Usage:", rm.get_cpu_usage())
    print("Memory Usage:", rm.get_memory_usage())
    print("Open Files:", rm.get_open_files())

    ma = MemoryAnalyzer()
    print("Memory Leaks:", ma.check_memory_leaks())
    print("Overflow Test:", ma.check_overflow(2**64))

    nta = NetworkTrafficAnalyzer()
    print("Network Stats:", nta.get_network_stats())

    sbc = SecurityBehaviorChecker()
    print("Suspicious Calls:", sbc.suspicious_calls())

    vs = VirtualSandbox()
    out, err = vs.run_isolated("echo Hello Sandbox")
    print("Sandbox stdout:", out)
    print("Sandbox stderr:", err)

```

- Code cho file "`app.py`":

dast-tool > app.py > ...

```python
from flask import Flask, jsonify, request
import os

from runtime_monitor import RuntimeMonitor
from memory_analyzer import MemoryAnalyzer
from network_traffic_analyzer import NetworkTrafficAnalyzer
from security_behavior_checker import SecurityBehaviorChecker
from virtual_sandbox import VirtualSandbox

app = Flask(__name__)
pid = os.getpid()

@app.route("/api/monitor", methods=["GET"])
def monitor():
    rm = RuntimeMonitor(pid)
    return jsonify({
        "cpu_usage": rm.get_cpu_usage(),
        "memory_usage": rm.get_memory_usage(),
        "open_files": [f.path for f in rm.get_open_files()]
    })

@app.route("/api/memory", methods=["GET"])
def memory():
    ma = MemoryAnalyzer()
    return jsonify({
        "memory_leaks": str(ma.check_memory_leaks()),
        "overflow_test": ma.check_overflow(2**64)
    })

@app.route("/api/network", methods=["GET"])
def network():
    nta = NetworkTrafficAnalyzer()
    return jsonify({
        "network_stats": {k: v._asdict() for k, v in nta.get_network_stats().items()}
    })

@app.route("/api/security", methods=["GET"])
def security():
    sbc = SecurityBehaviorChecker()
    return jsonify({
        "suspicious_calls": sbc.suspicious_calls()
    })
```

```python
@app.route("/api/sandbox", methods=["POST"])
def sandbox():
    data = request.json
    command = data.get("command")
    vs = VirtualSandbox()
    stdout, stderr = vs.run_isolated(command)
    return jsonify({
        "stdout": stdout,
        "stderr": stderr
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

```

- Code cho file “tests/test_memory_analyzer.py”:

dast-tool > tests > test_memory_analyzer.py > ...

```python
import unittest
from memory_analyzer import MemoryAnalyzer

class TestMemoryAnalyzer(unittest.TestCase):
    def test_check_overflow(self):
        ma = MemoryAnalyzer()
        self.assertTrue(ma.check_overflow(2**64))
        self.assertFalse(ma.check_overflow(100))

```

- Code cho file "tests/test_network_traffic_analyzer.py”:

dast-tool > tests > test_network_traffic_analyzer.py > ...

```python
import unittest
from network_traffic_analyzer import NetworkTrafficAnalyzer

class TestNetworkTrafficAnalyzer(unittest.TestCase):
    def test_get_network_stats(self):
        nta = NetworkTrafficAnalyzer()
        stats = nta.get_network_stats()
        self.assertIsInstance(stats, dict)

```

- Code cho file "tests/test_runtime_monitor.py”:

dast-tool > tests > test_runtime_monitor.py > ...

```python
import unittest
import os
from runtime_monitor import RuntimeMonitor

```

```python
class TestRuntimeMonitor(unittest.TestCase):
    def test_cpu_and_memory(self):
        rm = RuntimeMonitor(os.getpid())
        self.assertIsInstance(rm.get_cpu_usage(), float)
        self.assertGreaterEqual(rm.get_memory_usage(), 0)

```

- Code cho file "tests/test_security_behavior_checker.py”:

dast-tool > tests > test_security_behavior_checker.py > ...

```python
import unittest
from security_behavior_checker import SecurityBehaviorChecker

class TestSecurityBehaviorChecker(unittest.TestCase):
    def test_suspicious_calls(self):
        sbc = SecurityBehaviorChecker()
        self.assertIsInstance(sbc.suspicious_calls(), list)

```

- Code cho file "tests/test_virtual_sandbox.py”:

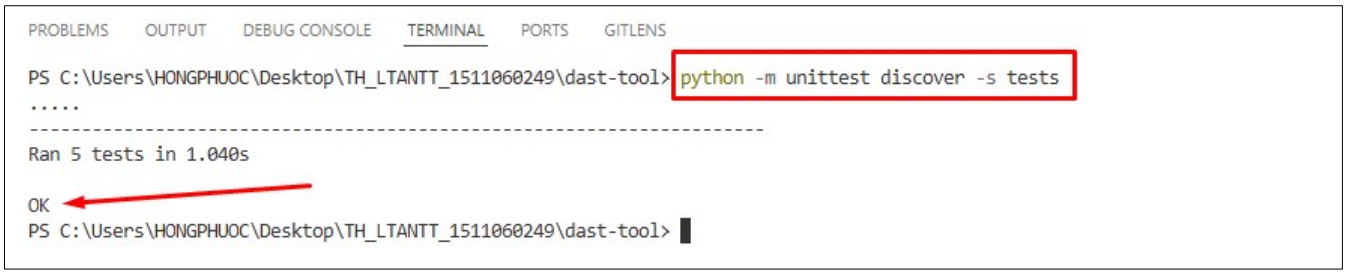
dast-tool > tests > test_virtual_sandbox.py > ...

```python
import unittest
from virtual_sandbox import VirtualSandbox

class TestVirtualSandbox(unittest.TestCase):
    def test_run_isolated(self):
        vs = VirtualSandbox()
        out, err = vs.run_isolated("echo test")
        self.assertIn("test", out)
        self.assertEqual(err, "")

```

- Chạy Unit Test:



- Chạy file “`main.py`”:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\dast-tool> python .\main.py
[+] Monitoring current process: 23580
CPU Usage: 0.0
Memory Usage: 17063936
Open Files: [popenfile(path='C:\\Windows\\System32\\en-US\\KernelBase.dll.mui', fd=-1)]
Memory Leaks: []
Overflow Test: True
Network Stats: {'Ethernet': snetio(bytes_sent=0, bytes_recv=0, packets_sent=0, packets_recv=0, errin=0, errout=0, dropin=0, dropout=0),
packets_recv=0, errin=0, errout=0, dropin=0, dropout=0), 'Local Area Connection* 2': snetio(bytes_sent=0, bytes_recv=0, packets_sent=0,
r VMnet1': snetio(bytes_sent=2100, bytes_recv=0, packets_sent=2100, packets_recv=0, errin=0, errout=0, dropin=0, dropout=0), 'VMware Ne
packets_recv=0, errin=0, errout=0, dropin=0, dropout=0), 'Wi-Fi': snetio(bytes_sent=645655950, bytes_recv=593623497, packets_sent=29427
ork Connection': snetio(bytes_sent=0, bytes_recv=0, packets_sent=0, packets_recv=0, errin=0, errout=0, dropin=0, dropout=0), 'Loopback
recv=0, errin=0, errout=0, dropin=0, dropout=0), 'vEthernet (Default Switch)': snetio(bytes_sent=3672155, bytes_recv=0, packets_sent=21
Suspicious Calls: ['os.system', 'eval', 'exec']
Sandbox stdout: Hello Sandbox

Sandbox stderr:
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\dast-tool>
```

- Giám sát với Flask API:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\dast-tool> python .\app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.17.68:5000
Press CTRL+C to quit
```

- Kiểm tra các API (lấy endpoint trong file app.py).

- Commit code, push code lên remote repo:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\dast-tool> git add .
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\dast-tool> git commit -m "[add] dast tool"
GitSecure: All checks passed.
[main 65df725] [add] dast tool
 14 files changed, 192 insertions(+)
 create mode 100644 dast-tool/__init__.py
 create mode 100644 dast-tool/app.py
 create mode 100644 dast-tool/main.py
 create mode 100644 dast-tool/memory_analyzer.py
 create mode 100644 dast-tool/network_traffic_analyzer.py
 create mode 100644 dast-tool/requirements.txt
 create mode 100644 dast-tool/runtime_monitor.py
 create mode 100644 dast-tool/security_behavior_checker.py
 create mode 100644 dast-tool/tests/test_memory_analyzer.py
 create mode 100644 dast-tool/tests/test_network_traffic_analyzer.py
 create mode 100644 dast-tool/tests/test_runtime_monitor.py
 create mode 100644 dast-tool/tests/test_security_behavior_checker.py
 create mode 100644 dast-tool/tests/test_virtual_sandbox.py
 create mode 100644 dast-tool/virtual_sandbox.py
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\dast-tool> git push origin main
Enumerating objects: 19, done.
Counting objects: 100% (19/19), done.
Delta compression using up to 8 threads
Compressing objects: 100% (16/16), done.
Writing objects: 100% (18/18), 3.62 KiB | 617.00 KiB/s, done.
Total 18 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/phuocntmh/TH_LTANTT_1511060249.git
   1632007..65df725  main -> main
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\dast-tool>
```